Eval：完结！

NoteKing | 图文讲义

Author / Channel	MiniMind / Mokio
Duration	8m17s
Generated by	NoteKing — Video to Illustrated Notes
Date	2026-04-03
Video Link	https://www.bilibili.com/video/BV1T2k6BaEeC
GitHub	github.com/bcefghj/noteking
Xiaohongshu	bcefghj

目录

1 模型评估、推理测试与展望 2

2 本讲概述 2

3 1. 模型评估基础 2

3.1 1.1 为什么需要模型评估 2

3.2 1.2 评估指标体系 3

3.2.1 1.2.1 语言模型的专用指标 3

3.2.2 1.2.2 任务相关指标 4

3.3 1.3 评估数据集划分 5

4 2. 推理测试详解 6

4.1 2.1 推理流程概述 6

4.2 2.2 模型加载与部署 7

4.3 2.3 文本生成推理 8

4.4 2.4 推理优化技术 10

5 3. 评估实践 11

5.1 3.1 标准评估流程 11

5.2 3.2 评估结果分析 13

6 4. 展望与未来方向 15

6.1 4.1 MiniMind 项目总结 15

6.2 4.2 未来优化方向 16

6.2.1 4.2.1 模型层面的改进 16

6.2.2 4.2.2 训练层面的改进 17

6.2.3 4.2.3 应用层面的扩展 17

6.3 4.3 学习路径建议 17

7 5. 本讲总结 18

8 关键要点 19

9 思考题 19

第 26 集：Eval：完结！

课程讲义：第二十六讲 Eval：完结！

1 模型评估、推理测试与展望

---

2 本讲概述

本讲是 MiniMind 系列的完结之作，将系统讲解大语言模型的评估方法、推理测试流程，并展望未来发展方向。作为课程的收尾，本讲将帮助学员建立完整的模型评估知识体系。

MINI Mind 本讲时长：8 分 17 秒 | 核心内容：模型评估体系、推理测试、展望/MINI Mind

---

3 1. 模型评估基础

3.1 1.1 为什么需要模型评估

图 1: Video 00:08 [1]

模型评估是机器学习工作流中不可或缺的环节，它决定了我们能否客观衡量模型的实际性能，并指导后续的

模型评估的核心目标包括：

[1] video timestamp 00:08

1. **量化性能**：用数值指标衡量模型的优劣
2. **对比基准**：与基线模型或其他方法进行比较
3. **问题诊断**：发现模型的不足之处
4. **指导优化**：为后续改进提供方向

3.2 1.2 评估指标体系

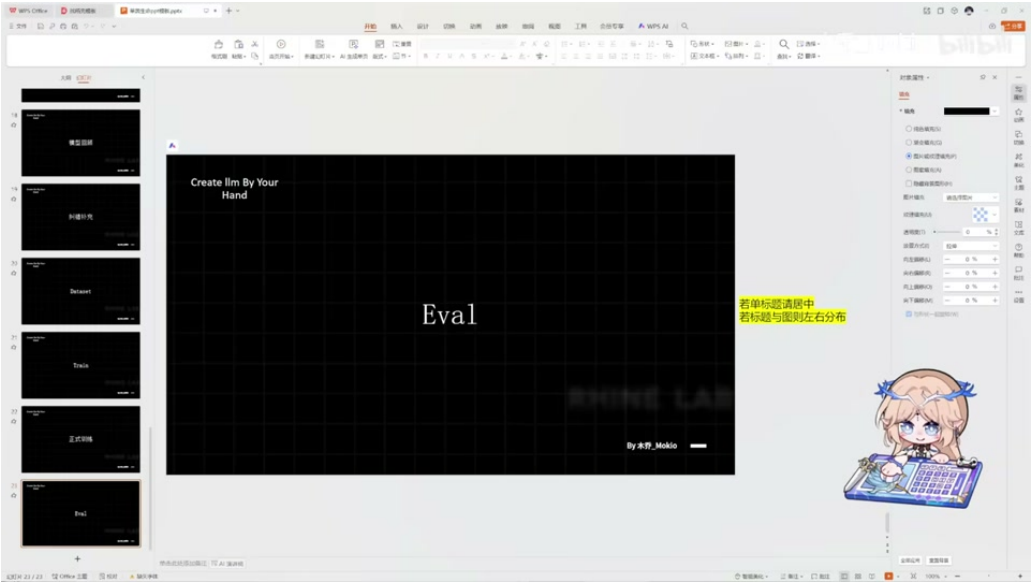

图 2: Video 00:09 [2]

对于语言模型，核心评估指标分为两类：生成质量指标和任务性能指标。

3.2.1 1.2.1 语言模型的专用指标

困惑度 (Perplexity)

困惑度是语言模型最基础的评价指标，定义为：

$$\text{Perplexity} = 2^{-\frac{1}{N}\sum_{i=1}^{N}\log_2 P(x_i|x_1,\ldots,x_{i-1})}$$

困惑度越低，表示模型对语言的建模能力越强。

Bits Per Character (BPC)

$$BPC = \frac{\text{Cross-Entropy}}{\ln(2)}$$

BPC 与困惑度可以相互转换，都是衡量语言建模能力的有效指标。

[2] video timestamp 00:09

3.2.2 1.2.2 任务相关指标

```python
# 常见任务评估指标示例
class EvaluationMetrics:
    """评估指标集合"""

    @staticmethod
    def accuracy(predictions, targets):
        """分类准确率"""
        return sum(p == t for p, t in zip(predictions, targets)) / len(targets)

    @staticmethod
    def f1_score(predictions, targets, num_classes):
        """F1分数"""
        from collections import Counter
        # 计算混淆矩阵元素
        tp = sum(1 for p, t in zip(predictions, targets) if p == t and p == 1)
        fp = sum(1 for p, t in zip(predictions, targets) if p == 1 and t == 0)
        fn = sum(1 for p, t in zip(predictions, targets) if p == 0 and t == 1)

        precision = tp / (tp + fp) if (tp + fp) > 0 else 0
        recall = tp / (tp + fn) if (tp + fn) > 0 else 0

        f1 = 2 * precision * recall / (precision + recall) if (precision + recall) > 0 else 0
        return f1

    @staticmethod
    def perplexity(loss, base=2):
        """根据交叉熵损失计算困惑度"""
        import math
        return math.pow(base, loss)
```

3.3 1.3 评估数据集划分

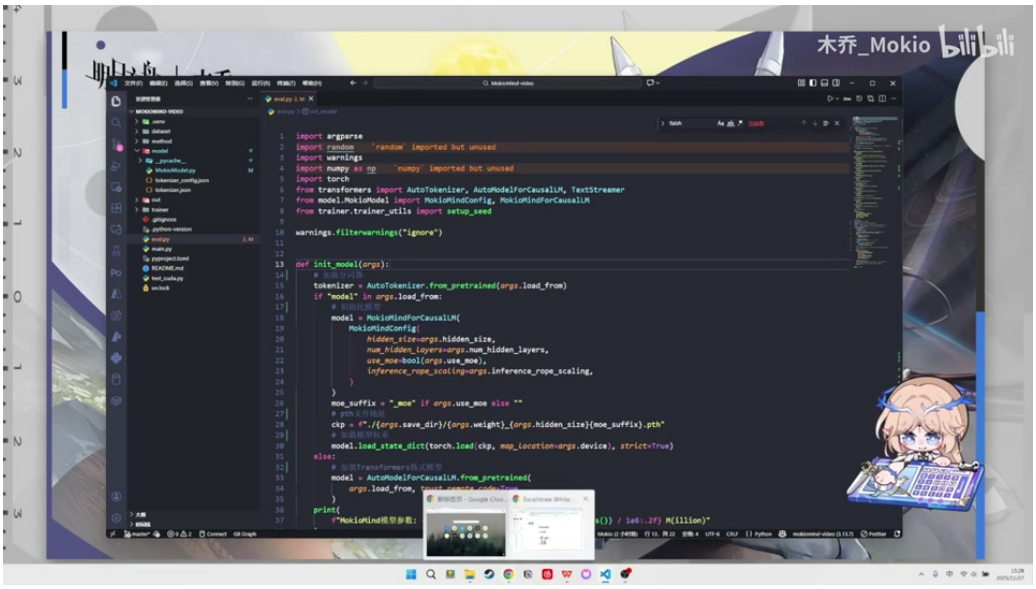

图 3: Video 00:31 [3]

数据集划分是避免过拟合和获得可靠评估结果的关键。常见划分比例为训练集: 验证集: 测试集 = 8:1:1。

```python
def split_dataset(data, train_ratio=0.8, val_ratio=0.1, seed=42):
    """数据集划分函数"""
    import random
    random.seed(seed)
    data_copy = data.copy()
    random.shuffle(data_copy)

    n = len(data_copy)
    train_end = int(n * train_ratio)
    val_end = int(n * (train_ratio + val_ratio))

    return {
        'train': data_copy[:train_end],
        'val': data_copy[train_end:val_end],
        'test': data_copy[val_end:]
    }
```

[3] video timestamp 00:31

4 2. 推理测试详解

4.1 2.1 推理流程概述

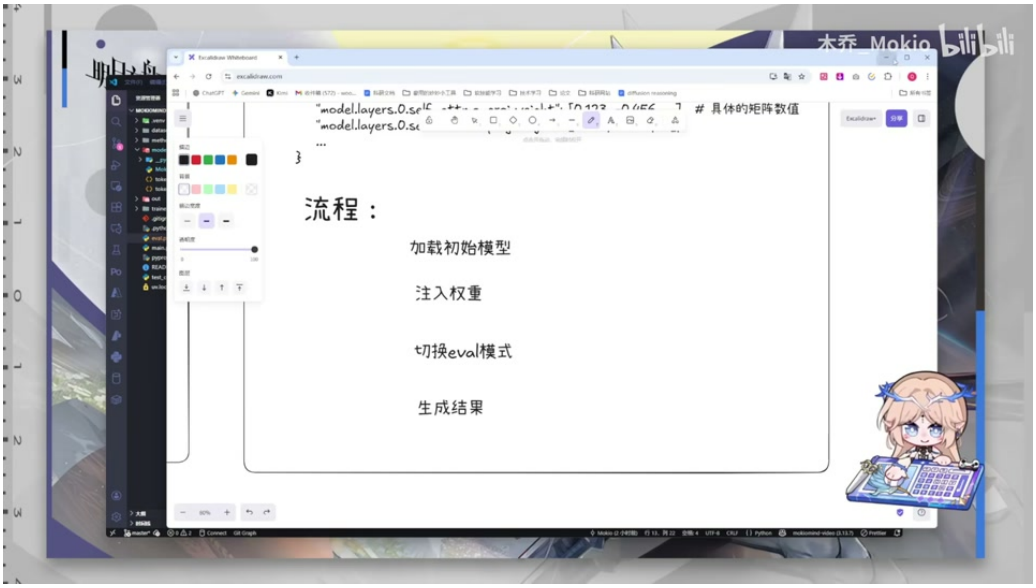

图 4: Video 02:01 [4]

推理（Inference）是使用训练好的模型对新数据进行预测的过程。与训练不同，推理通常只需要前向传播，不需要反向传播。

完整的推理流程包括：

1. 模型加载
2. 输入预处理
3. 分词处理
4. 模型前向传播
5. 结果后处理
6. 解码输出

[4] video timestamp 02:01

4.2 2.2 模型加载与部署

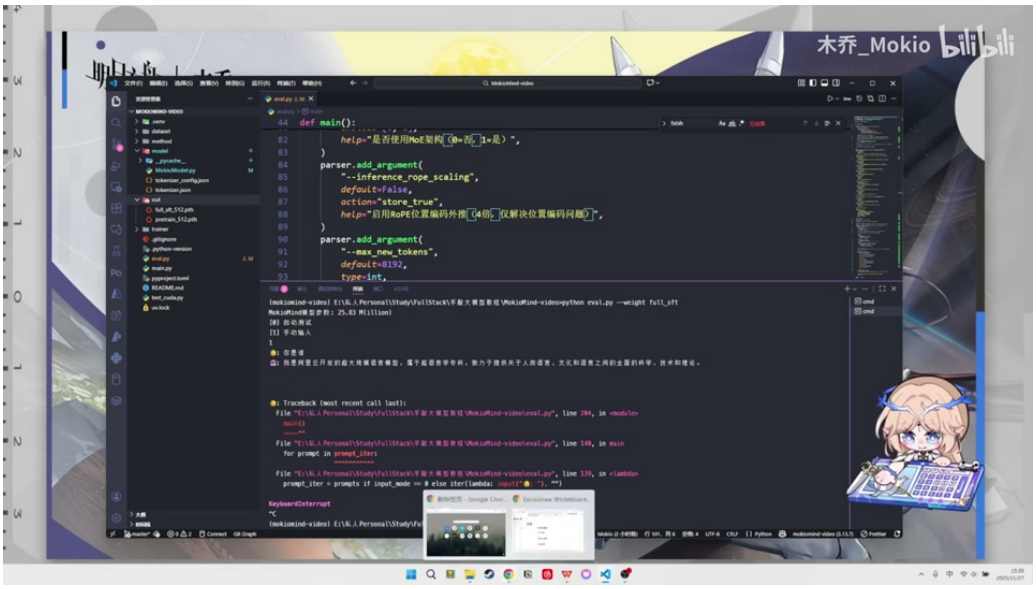

图 5: Video 07:29 [5]

```python
import torch
from minilm import MiniLM

def load_model_for_inference(checkpoint_path, config):
    """加载模型用于推理"""
    # 设置设备
    device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

    # 创建模型实例
    model = MiniLM(
        vocab_size=config['vocab_size'],
        hidden_size=config['hidden_size'],
        num_layers=config['num_layers'],
        num_heads=config['num_heads'],
        max_seq_length=config['max_seq_length']
    )

    # 加载权重
    checkpoint = torch.load(checkpoint_path, map_location=device)
    model.load_state_dict(checkpoint['model_state_dict'])

    # 切换到评估模式
    model.eval()
    model.to(device)

```

[5] video timestamp 07:29

```
    return model, device
```

4.3 2.3 文本生成推理

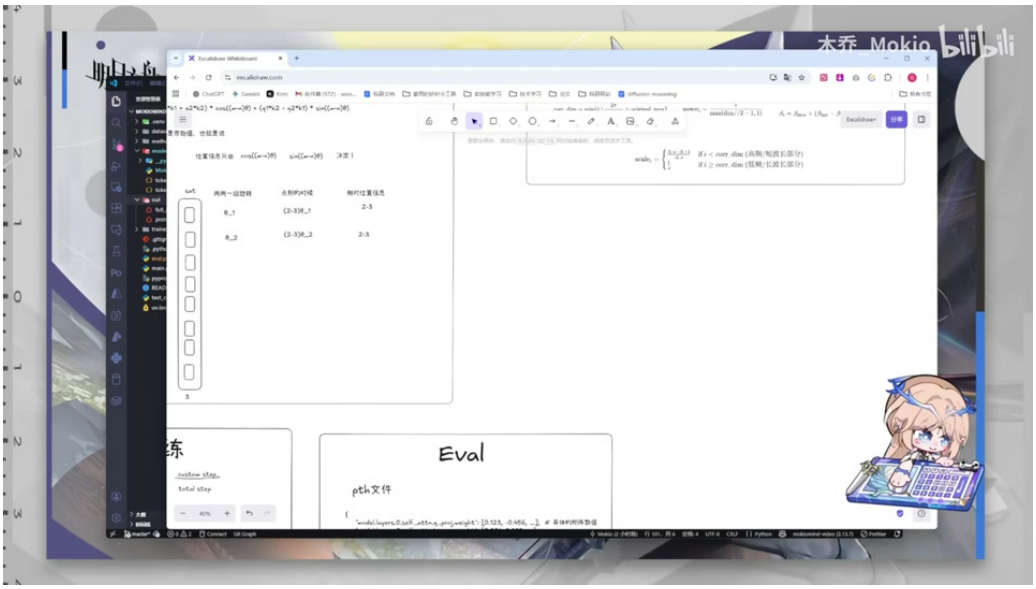

图 6: Video 07:51 [6]

自回归语言模型的核心是逐 token 生成，每个 token 的生成都依赖于之前所有的 token。

```
def generate_text(model, tokenizer, prompt, max_length=100, temperature=0.8, top_k=50):
    """
    文本生成推理函数

    Args:
        model: 训练好的语言模型
        tokenizer: 分词器
        prompt: 输入提示
        max_length: 最大生成长度
        temperature: 采样温度（控制随机性）
        top_k: Top-K采样参数
    """
    device = next(model.parameters()).device

    # Tokenize 输入
    input_ids = tokenizer.encode(prompt)
    input_tensor = torch.tensor([input_ids], device=device)

    # 自回归生成
```

[6] video timestamp 07:51

```python
    generated_ids = input_ids.copy()

    for _ in range(max_length):
        # 前向传播
        with torch.no_grad():
            logits = model(input_tensor)

        # 获取最后一个位置的logits
        next_token_logits = logits[0, -1, :] / temperature

        # Top-K 过滤
        if top_k > 0:
            indices_to_remove = next_token_logits < torch.topk(
                next_token_logits, top_k)[0][-1]
            next_token_logits[indices_to_remove] = float('-inf')

        # 转换为概率并采样
        probs = torch.softmax(next_token_logits, dim=-1)
        next_token = torch.multinomial(probs, num_samples=1).item()

        # 检查是否生成结束符
        if next_token == tokenizer.eos_token_id:
            break

        # 追加到生成序列
        generated_ids.append(next_token)
        input_tensor = torch.tensor([generated_ids], device=device)

    return tokenizer.decode(generated_ids)
```

4.4 2.4 推理优化技术

图 7: Video 08:16 [7]

推理优化可以显著提升模型部署效率，常见技术包括：量化、剪枝、蒸馏和批处理。

```python
# 推理优化示例
class InferenceOptimizer:
    """推理优化器"""

    @staticmethod
    def apply_quantization(model, dtype=torch.qint8):
        """动态量化"""
        model_quantized = torch.quantization.quantize_dynamic(
            model, {torch.nn.Linear}, dtype=dtype
        )
        return model_quantized

    @staticmethod
    def prepare_for_mobile(model):
        """准备移动端部署"""
        model.qconfig = torch.quantization.get_default_qconfig('qnnpack')
        torch.quantization.prepare(model, inplace=True)
        torch.quantization.convert(model, inplace=True)
        return model

    @staticmethod
    def batch_inference(model, input_batch):
        """批量推理以提高吞吐量"""
```

[7] video timestamp 08:16

```
        with torch.no_grad():
            outputs = model(input_batch)
        return outputs
```

5 3. 评估实践

5.1 3.1 标准评估流程

图 8: Video 00:03 [8]

一个完整的模型评估流程应该包括：加载模型 → 准备数据 → 执行评估 → 记录结果 → 生成报告。

```
class ModelEvaluator:
    """模型评估器"""

    def __init__(self, model, tokenizer, device):
        self.model = model
        self.tokenizer = tokenizer
        self.device = device

    def evaluate_perplexity(self, dataset):
        """评估困惑度"""
        self.model.eval()
        total_loss = 0
```

[8] video timestamp 00:03

```python
        total_tokens = 0

        with torch.no_grad():
            for batch in dataset:
                input_ids = batch['input_ids'].to(self.device)
                labels = batch['labels'].to(self.device)

                outputs = self.model(input_ids, labels=labels)
                loss = outputs.loss

                num_tokens = (labels != -100).sum().item()
                total_loss += loss.item() * num_tokens
                total_tokens += num_tokens

        avg_loss = total_loss / total_tokens
        perplexity = 2 ** avg_loss if total_tokens > 0 else float('inf')

        return {
            'avg_loss': avg_loss,
            'perplexity': perplexity,
            'total_tokens': total_tokens
        }

    def evaluate_downstream_tasks(self, task_datasets):
        """评估下游任务性能"""
        results = {}

        for task_name, (dataset, metric_fn) in task_datasets.items():
            predictions = []
            references = []

            self.model.eval()
            with torch.no_grad():
                for batch in dataset:
                    outputs = self.model(batch['input_ids'].to(self.device))
                    preds = outputs.logits.argmax(dim=-1)
                    predictions.extend(preds.cpu().tolist())
                    references.extend(batch['labels'].tolist())

            results[task_name] = metric_fn(predictions, references)

        return results

    def run_full_evaluation(self, test_data, task_datasets=None):
        """运行完整评估"""
        print("=" * 50)
```

```python
        print("开始模型评估")
        print("=" * 50)

        # 1. 困惑度评估
        perplexity_results = self.evaluate_perplexity(test_data)
        print(f"\n困惑度评估结果:")
        print(f"  - 平均损失: {perplexity_results['avg_loss']:.4f}")
        print(f"  - 困惑度: {perplexity_results['perplexity']:.2f}")

        # 2. 下游任务评估
        if task_datasets:
            task_results = self.evaluate_downstream_tasks(task_datasets)
            print(f"\n下游任务评估结果:")
            for task_name, score in task_results.items():
                print(f"  - {task_name}: {score:.4f}")

        print("\n" + "=" * 50)
        print("评估完成")
        print("=" * 50)

        return {
            'perplexity': perplexity_results,
            'downstream': task_results if task_datasets else None
        }
```

5.2 3.2 评估结果分析

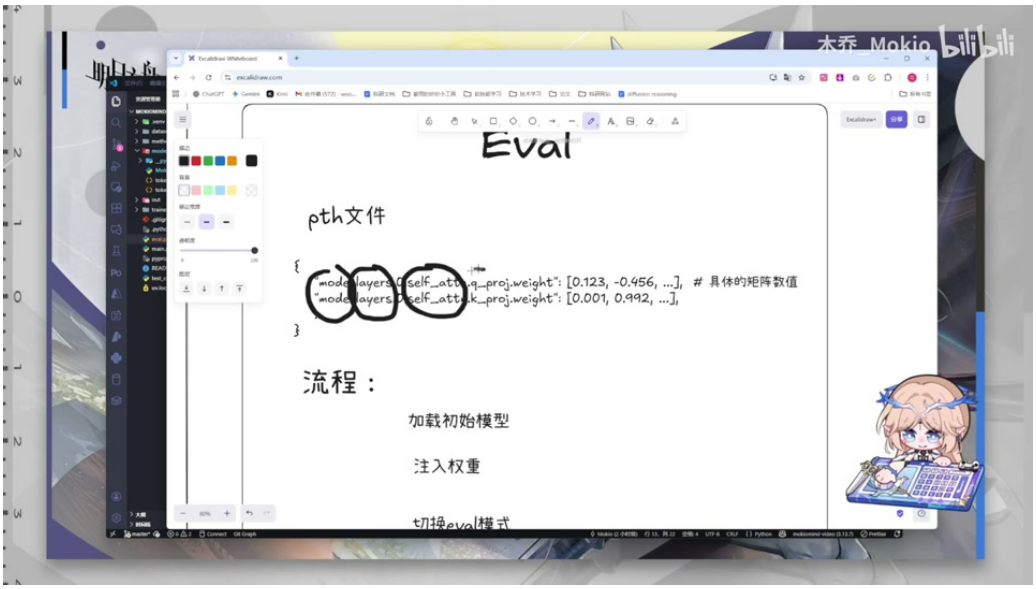

图 9: Video 00:58 [9]

[9] video timestamp 00:58

评估结果的分析需要结合多个指标综合判断，单一指标往往不能完整反映模型能力。

```python
def analyze_evaluation_results(results, baseline_results=None):
    """分析评估结果"""
    analysis = {
        'summary': {},
        'comparisons': {},
        'recommendations': []
    }

    # 与基线模型对比
    if baseline_results:
        analysis['comparisons']['perplexity_improvement'] = (
            baseline_results['perplexity'] - results['perplexity']
        ) / baseline_results['perplexity'] * 100

        # 生成对比报告
        print(f"\n与基线模型对比:")
        print(f"  困惑度改善: {analysis['comparisons']['perplexity_improvement']:.2f}%")

    # 性能评级
    perplexity = results['perplexity']
    if perplexity < 20:
        grade = "优秀"
    elif perplexity < 40:
        grade = "良好"
    elif perplexity < 60:
        grade = "中等"
    else:
        grade = "待改进"

    analysis['summary']['grade'] = grade
    analysis['summary']['perplexity'] = perplexity

    return analysis
```

6 4. 展望与未来方向

6.1 4.1 MiniMind 项目总结

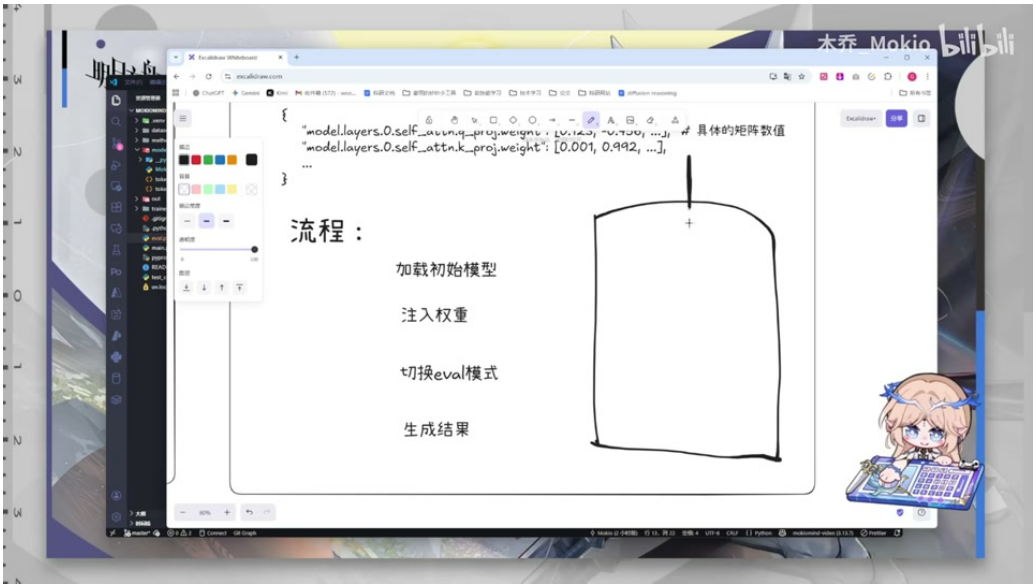

图 10: Video 01:25 [10]

MiniMind 项目从零开始实现了一个完整的大语言模型，涵盖了数据处理、模型架构、训练流程和模型评

项目关键里程碑：

阶段	内容	核心收获
数据处理	语料清洗、分词、Tokenization	数据质量决定模型上限
模型架构	Transformer 实现	理解现代 LLM 核心结构
训练优化	分布式训练、混合精度	大模型训练的工程实践
模型评估	困惑度、任务评估	科学评估模型能力

[10] video timestamp 01:25

6.2 4.2 未来优化方向

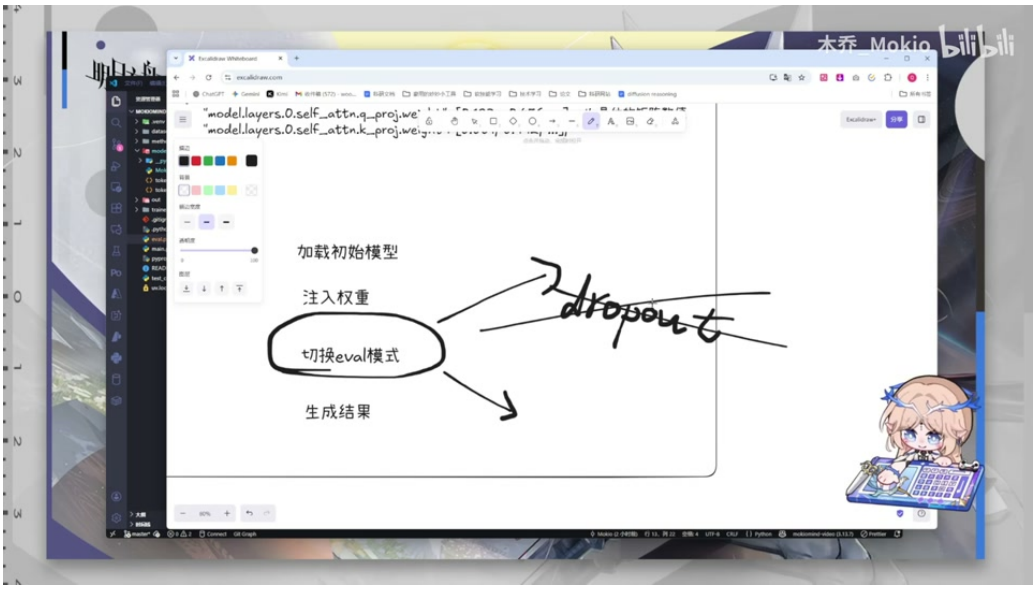

图 11: Video 01:53 [11]

模型优化是一个持续迭代的过程，需要在多个维度进行改进。

6.2.1 4.2.1 模型层面的改进

1. **模型规模放大**：增加层数、隐藏维度、注意力头数
2. **架构创新**：探索新的注意力机制、位置编码方式
3. **知识蒸馏**：从大模型提取知识到小模型

```python
# 模型规模扩展示例
class ScalingConfig:
    """模型扩展配置"""

    # 从 Mini 模型到 Large 模型的扩展
    MINI_CONFIG = {
        'hidden_size': 256,
        'num_layers': 6,
        'num_heads': 8,
        'ffn_dim': 512,
        'max_seq_length': 512
    }

    MEDIUM_CONFIG = {
        'hidden_size': 512,
```

[11] video timestamp 01:53

```
        'num_layers': 12,
        'num_heads': 8,
        'ffn_dim': 2048,
        'max_seq_length': 1024
    }

    LARGE_CONFIG = {
        'hidden_size': 1024,
        'num_layers': 24,
        'num_heads': 16,
        'ffn_dim': 4096,
        'max_seq_length': 2048
    }
```

6.2.2 4.2.2 训练层面的改进

1. **更多高质量数据**：扩大预训练语料库
2. **课程学习**：按难度渐进训练
3. **RLHF**：引入人类反馈强化学习

6.2.3 4.2.3 应用层面的扩展

1. **多模态能力**：结合视觉、语音等模态
2. **工具使用**：集成外部 API 和工具调用
3. **长上下文**：支持更长的输入序列

6.3 4.3 学习路径建议

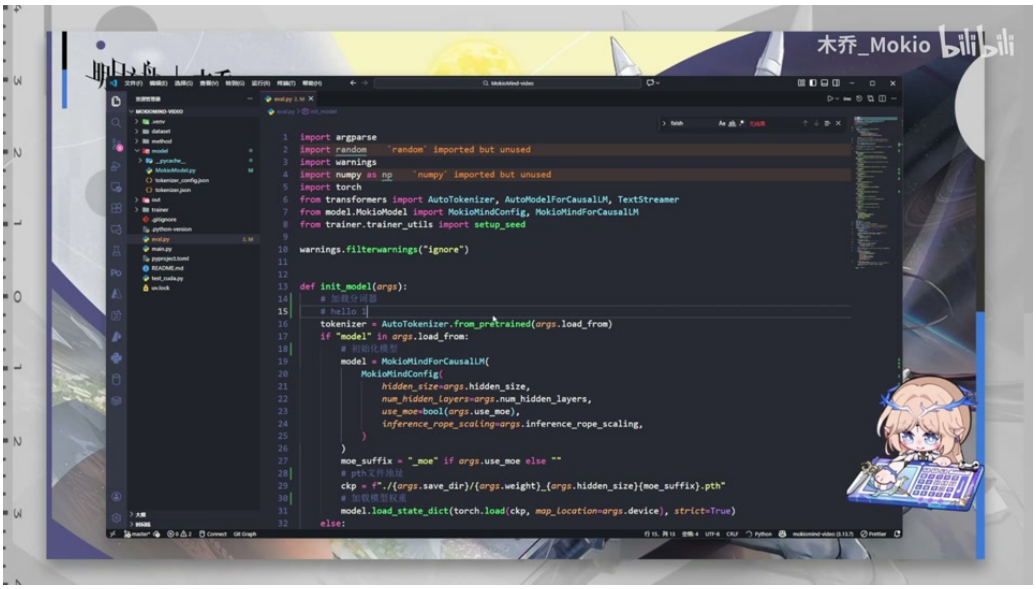

图 12: Video 02:20 [12]

持续学习和实践是掌握大模型技术的关键。

推荐学习路径：

```
阶段一：基础夯实
    深入理解 Transformer 架构
    掌握 PyTorch 高级用法
    学习分布式训练原理

阶段二：进阶提升
    研究预训练语言模型原理
    实现模型评估与分析
    参与开源项目贡献

阶段三：前沿探索
    追踪最新论文和研究
    探索模型压缩与优化
    实践 RLHF 和 Agent
```

---

7 5. 本讲总结

本讲作为 MiniMind 系列的收官之作，系统介绍了大语言模型的评估方法和推理测试流程。主要内容包括：

1. **模型评估基础**：介绍了困惑度、BPC 等核心指标，以及数据集划分方法
2. **推理测试详解**：讲解了完整的推理流程，包括模型加载、文本生成和优化技术
3. **评估实践**：提供了完整的评估代码实现和结果分析方法
4. **未来展望**：总结了项目成果并指出了可能的改进方向

模型评估是机器学习的核心环节，掌握科学的评估方法对于模型优化至关重要。

---

[12] video timestamp 02:20

8 关键要点

- **困惑度 (Perplexity)** 是衡量语言模型建模能力的核心指标，越低越好
- **推理优化**可以显著提升模型部署效率，包括量化、剪枝、批处理等技术
- **综合评估**需要结合多个指标，避免单一指标导致的误导
- **持续迭代**是模型优化的核心，需要不断尝试新的方法和技巧

---

9 思考题

1. **为什么困惑度不是评估语言模型的唯一指标？**思考在实际应用中，还有哪些维度需要评估?
2. **假设你要优化 MiniMind 模型，你会从哪些方向入手？**请结合本系列课程所学知识，制定一个优化计划。

课后作业：
使用本讲提供的评估代码，对 MiniMind 模型进行完整评估，并生成一份评估报告。